

Student Mental Health Monitoring System

SE LAB Experiment-08 – Part A

System Design Document (SDD)

Architecture, Subsystems, Hardware Mapping, Data Management & Control Flow

Submitted By:

Mokshi Shah
241071068

Vanshika Shah
241071069

April 21, 2026

Contents

1	Design Goals	2
1.1	Functional Goals	2
1.2	Non-Functional Goals	3
1.3	Trade-offs and Priority	4
2	Subsystem Decomposition	5
2.1	Overview of Subsystem Decomposition Diagram	5
2.2	Class Diagram for Each Subsystem	5
2.2.1	SS-1: Authentication & Session Subsystem	5
2.2.2	SS-2: Screening & Logging Subsystem	6
2.2.3	SS-3: Analytics & Risk Engine Subsystem	6
2.2.4	SS-4: Notification Subsystem	7
2.2.5	SS-5: Admin & Reporting Subsystem	7
3	Hardware/Software Mapping	8
3.1	Deployment Diagram	8
3.2	Hardware Nodes and Software Artifacts	9
4	Data Management	10
4.1	Persistent Data Entities	10
4.2	Non-Persistent or Cached Data	11
4.3	Data Retention and Backup Policies	12
5	Role-Based Access Control	13
6	Control Flow Mechanism	14
6.1	Primary Control Patterns	14
6.2	Workflow Flowcharts	15
6.2.1	Workflow 1: Student Screening and Risk Classification	15
6.2.2	Workflow 2: High-Risk Alert and Counsellor Response	16
6.2.3	Workflow 3: Admin Institutional Report Generation	17
6.3	Concurrency and Locking	18
7	Boundary Conditions	19
7.1	System Startup	19
7.2	System Shutdown	19
7.3	Exceptional Input Handling	20
8	Conclusion	21

1 DESIGN GOALS

Design goals direct architectural and implementation decisions throughout the SMHMS system. They are derived from the system requirements and stakeholder priorities. All goals are classified by type and assigned a priority (High / Medium / Low).

1.1 Functional Goals

#	Goal	Description
FG-1	Early Risk Detection	The system must proactively detect Low/Medium/High mental health risk using PHQ-9, GAD-7, PSS scores combined with academic and behavioural data.
FG-2	Automated Alert Delivery	High-risk classifications must trigger real-time alerts to the assigned counsellor and admin within 2 minutes via in-app and email channels.
FG-3	Role-Based Dashboards	Each actor (Student, Counsellor, Admin, Guardian, IT Officer) must see a personalised, role-filtered dashboard with no cross-role data leakage.
FG-4	Session Lifecycle Management	Counsellors must be able to schedule, reschedule, and cancel sessions with automated 24-hour reminders and immutable session history.
FG-5	Institutional Trend Reporting	Admins must generate fully anonymised institutional trend reports exportable as PDF and CSV on weekly, monthly, or on-demand schedules.
FG-6	Consent-Gated Guardian Access	Parent/Guardian access must be strictly consent-gated; revocation must propagate immediately to all endpoints and be logged in the audit trail.
FG-7	SIS Integration	The system must pull academic and attendance data from the institutional SIS weekly via read-only API; CSV import must function as a fallback.

1.2 Non-Functional Goals

#	Category	Goal	Priority	Measurable Criterion
NFG1	Performance	Dashboard response time	High	≤ 2 sec at 200 concurrent users (Apache JMeter)
NFG2	Performance	Risk classification latency	High	Single student ≤ 5 sec; full batch ≤ 2 hrs
NFG3	Security	Data encryption	High	AES-256 at rest; TLS 1.3 in transit; bcrypt(12) for passwords
NFG4	Security	Brute-force protection	High	Account locked after 5 failed logins; logged to AuditLog
NFG5	Availability	System uptime	High	$\geq 99.5\%$ uptime (≤ 44 hrs unplanned downtime/year)
NFG6	Scalability	Horizontal scalability	Medium	Containerised (Docker); auto-scale via Docker Compose replicas
NFG7	Usability	Onboarding speed	Medium	New student completes first screening in ≤ 5 min unaided
NFG8	Accessibility	WCAG compliance	Medium	WCAG 2.1 Level AA; tested with axe-core before deployment
NFG9	Maintainability	Test coverage	Medium	$\geq 80\%$ code coverage (pytest); all PRs peer-reviewed
NFG10	Compliance	GDPR / data privacy	High	Right-to-erasure pipeline; consent audit trail; minimal data exposure

1.3 Trade-offs and Priority

Trade-off	Option A (chosen)	Option B (rejected)
Security vs. Convenience	Mandatory 2FA for Admin/IT Officer even if slower login	Optional 2FA for all roles to maximise convenience
Accuracy vs. Latency	ML scorer runs on weekly batch (high accuracy, delayed)	ML scorer on every submission (real-time but high CPU load)
Isolation vs. Resource Use	Separate containers per service (Nginx, Flask, PostgreSQL, Redis)	Single monolithic container (simpler but no fault isolation)
Normalisation vs. Query Speed	Partially denormalised <code>RiskProfile.trendData</code> (JSON) for fast reads	Fully normalised score history table (consistent but slower joins)
Full Audit vs. Storage Cost	Append-only audit log with HMAC hash chain (tamper-evident)	Overwrite-on-update audit records (cheaper but not tamper-proof)
Guardian Access vs. Privacy	Consent-gated summary view only; no raw clinical data	Full profile access for guardians (higher parental visibility)

Priority Stack: Security (NFG-3/4/10) > Risk Detection Accuracy (FG-1/2) > Performance (NFG-1/2) > Usability (NFG-7/8) > Scalability (NFG-6) > Maintainability (NFG-9).

2 SUBSYSTEM DECOMPOSITION

SMHMS follows a layered, service-oriented architecture decomposed into five functional subsystems. Each subsystem owns a cohesive set of use cases and communicates with others through well-defined interfaces.

2.1 Overview of Subsystem Decomposition Diagram

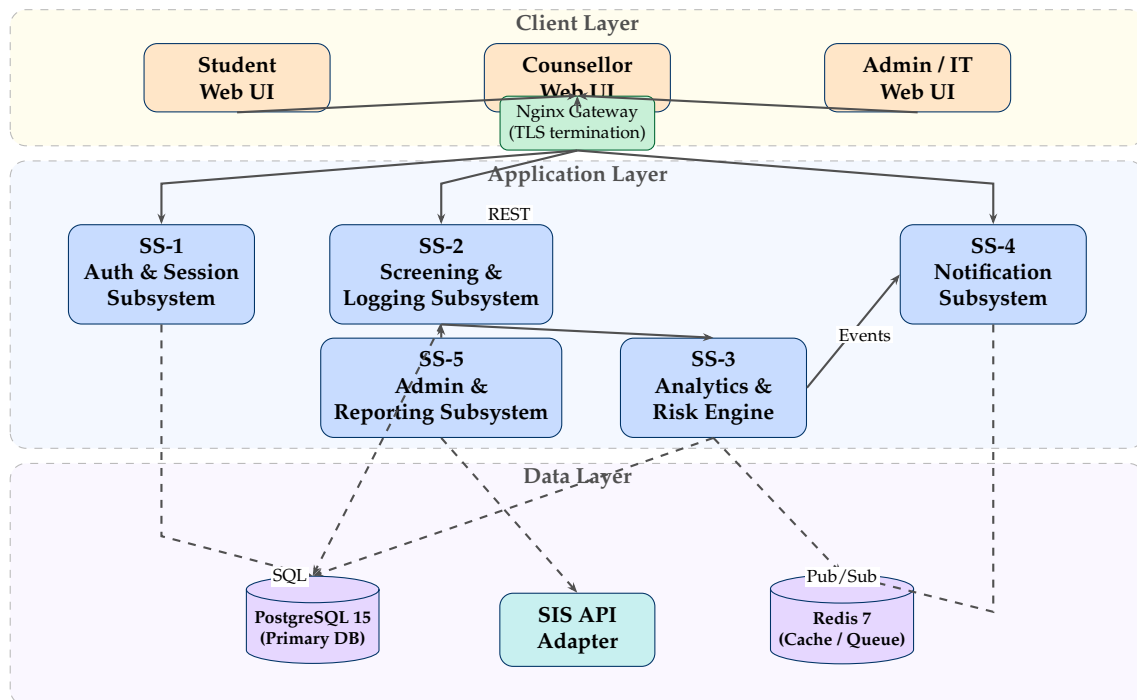


Figure 1: SMHMS Subsystem Decomposition – Five Subsystems across Three Layers

2.2 Class Diagram for Each Subsystem

2.2.1 SS-1: Authentication & Session Subsystem

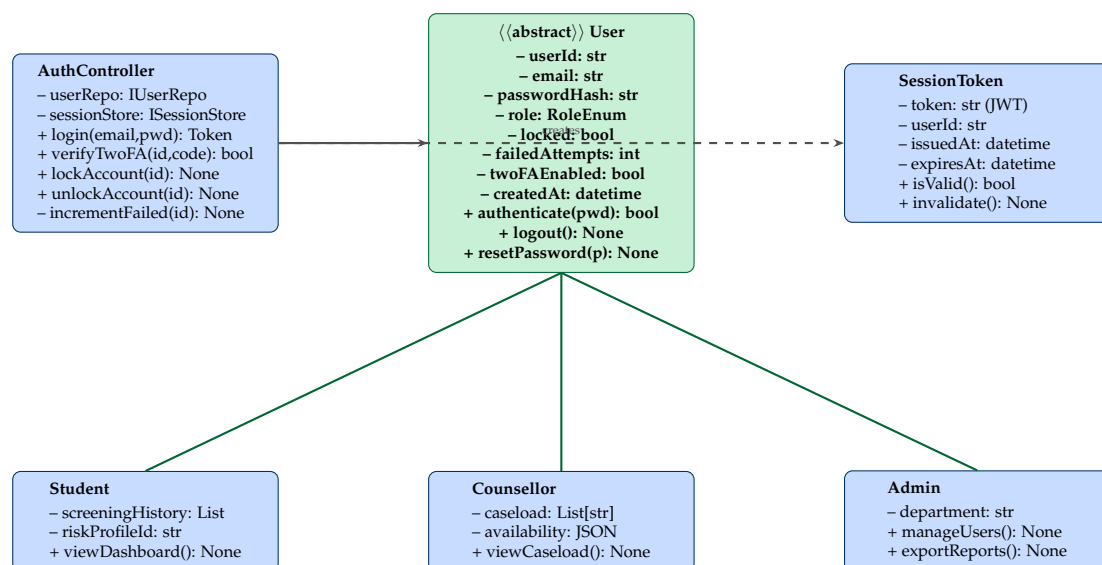


Figure 2: SS-1: Auth & Session Subsystem Class Diagram

2.2.2 SS-2: Screening & Logging Subsystem

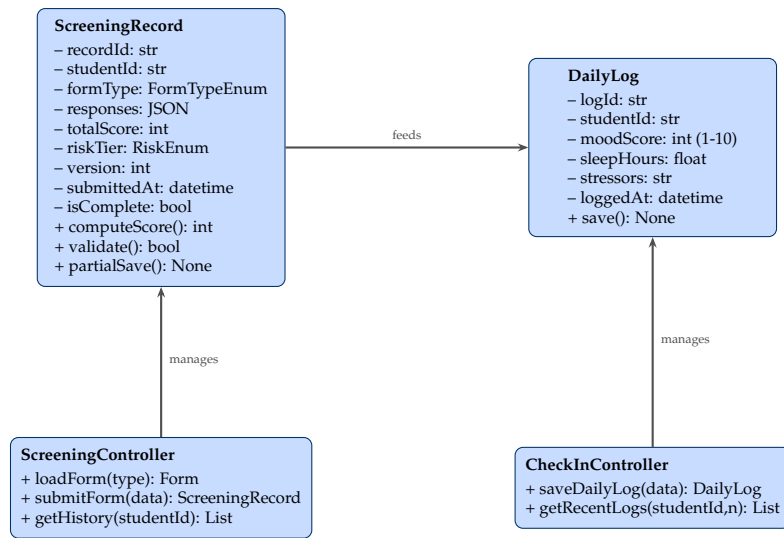


Figure 3: SS-2: Screening & Logging Subsystem Class Diagram

2.2.3 SS-3: Analytics & Risk Engine Subsystem

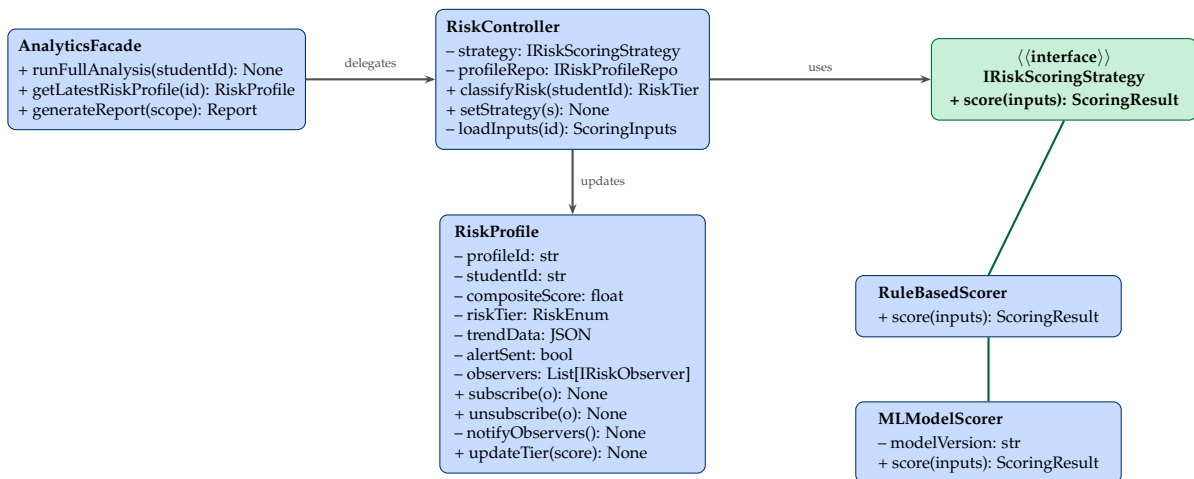


Figure 4: SS-3: Analytics & Risk Engine Class Diagram (Strategy + Observer + Facade)

2.2.4 SS-4: Notification Subsystem

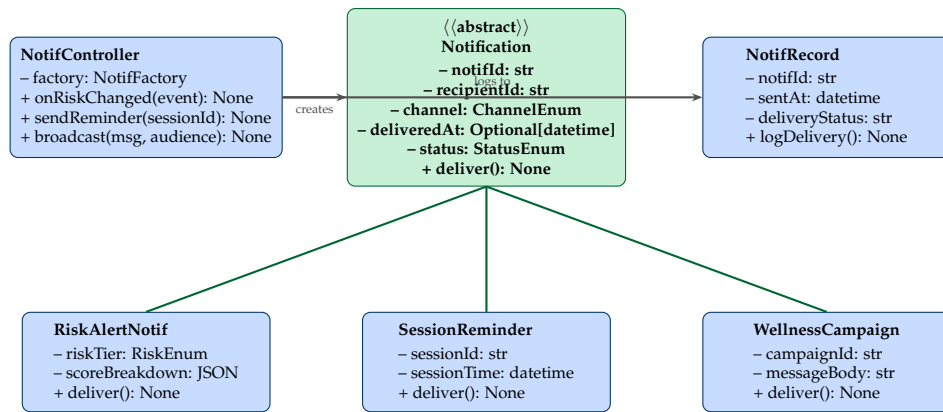


Figure 5: SS-4: Notification Subsystem Class Diagram

2.2.5 SS-5: Admin & Reporting Subsystem

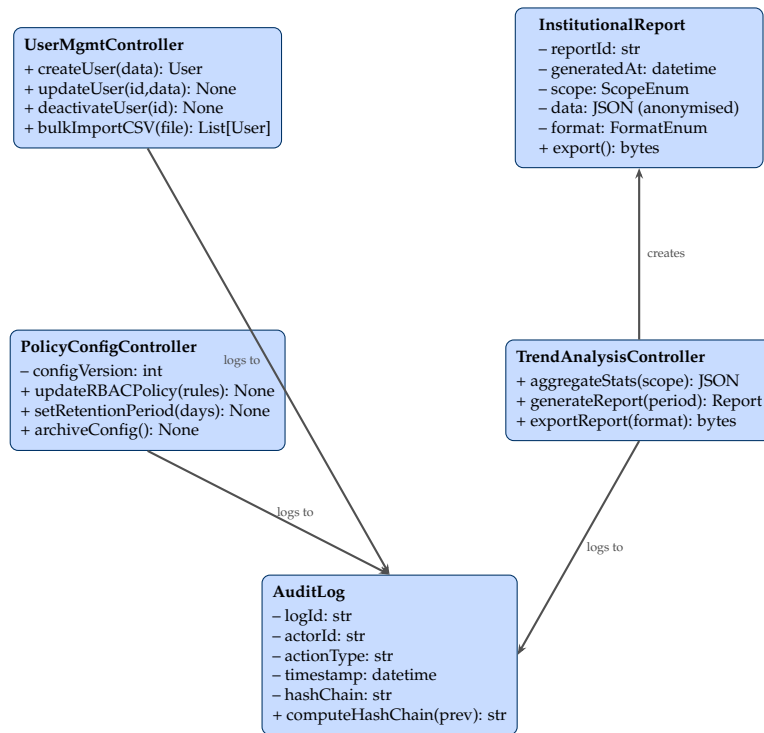


Figure 6: SS-5: Admin & Reporting Subsystem Class Diagram

3 HARDWARE/SOFTWARE MAPPING

3.1 Deployment Diagram

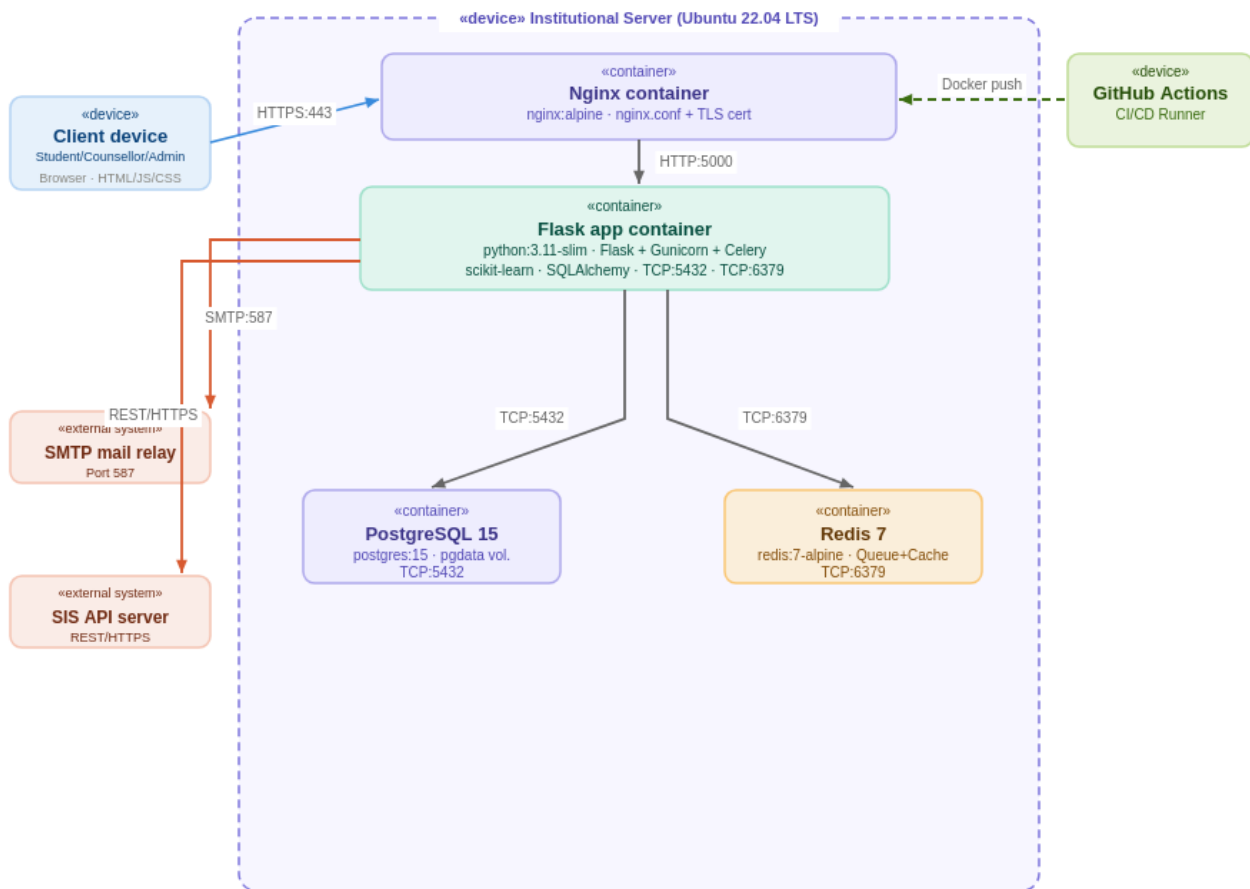


Figure 7: SMHMS Deployment Diagram – Containerised Institutional Server Architecture

3.2 Hardware Nodes and Software Artifacts

Node	Type	Specification
Institutional Server	Physical / VM	Ubuntu 22.04 LTS; 8 vCPU; 16 GB RAM; 500 GB SSD; Docker 24 + Docker Compose 2; managed by institution IT
Client Device	End-user device	Any modern browser (Chrome 110+, Firefox 110+, Safari 16+); no local installation required; mobile-responsive from 360px
SMTP Mail Relay	External SaaS	Institutional email server or SendGrid API; TLS-encrypted outbound on port 587
SIS API Server	Institutional system	Existing Student Information System; read-only REST API; HTTPS; institutional intranet
CI/CD Runner	GitHub Actions	Ubuntu 22.04 runners; Docker build + push pipeline; automated test + lint on every pull request

Container	Image	Key Artifacts	Responsibility
Nginx Container	nginx:alpine	nginx.conf, TLS certificates (Let's Encrypt), dhparam.pem	TLS termination, reverse proxy to Flask on :5000, static file serving
Flask App Container	python:3.11-slim	app.py, requirements.txt, gunicorn.conf.py, celery_worker.py, ML model .pkl files	Business logic, REST API endpoints, Celery task execution, ML inference
PostgreSQL 15	postgres:15	init.sql, smhms_pgdata Docker volume, pg_hba.conf	Persistent relational data storage; all domain entities
Redis 7	redis:7-alpine	redis.conf (persistence disabled), AOF log (session data only)	Celery task queue, JWT session cache (30-min TTL), rate-limit counters

4 DATA MANAGEMENT

4.1 Persistent Data Entities

Entity / Table	Primary Key	Key Fields and Notes
users	userId (UUID)	email (unique), passwordHash, role, locked, failedAttempts, twoFAEnabled, createdAt, lastLoginAt. Soft-delete only.
screening_record	recordId (UUID)	studentId (FK), formType, responses (JSONB), totalScore, riskTier, version, submittedAt, isComplete. Immutable after isComplete=True.
daily_logs	logId (UUID)	studentId (FK), moodScore (1–10), sleepHours, stressors (text), loggedAt (UTC). Retained for 24 months.
risk_profiles	profileId (UUID)	studentId (unique FK), compositeScore, riskTier, trendData (JSONB, max 52 entries), classifiedAt, alertSent. One per student.
counselling_sessions	sessionId (UUID)	studentId, counsellorId (both FK → users), slotDateTime, status (state machine), sessionNotes (encrypted), reminderSent, createdAt.
notifications	notifId (UUID)	recipientId (FK), channel, type, payload (JSONB), sentAt, deliveryStatus. Retained for 6 months.
consent_records	consentId (UUID)	studentId, guardianId (both FK), granted (bool), grantedAt, revokedAt (nullable). One active record per (student, guardian) pair.
audit_logs	logId (UUID)	actorId, actionType, targetId, targetTable, timestamp (UTC), hashChain (HMAC-SHA256). Append-only; no UPDATE/DELETE.
system_config	configId (UUID)	configKey, configValue (JSONB), version (int), appliedAt, appliedBy (FK → users). Previous versions archived.
rbac_policies	policyId (UUID)	role, resource, operations (JSONB array), createdAt, validFrom.

4.2 Non-Persistent or Cached Data

Data Item	Store	TTL	Purpose
JWT session tokens	Redis	30 min (inactivity)	Fast session validation without DB hit on every request
Rate-limit counters	Redis	15 min window	Brute-force protection; max 5 failed logins per window per IP
Celery task queue	Redis	Until consumed	Async task dispatch: alert emails, report generation, reminder scheduling
Analytics cache	Redis	5 min	Cached dashboard KPIs; invalidated on new risk classification
Screening form drafts	Redis	60 min	Temporary partial-save state before full submission to PostgreSQL
TOTP 2FA codes	Redis	30 sec	One-time codes for admin/IT login; auto-expired

4.3 Data Retention and Backup Policies

Data Category	Retention Period	Backup Frequency	Notes
Screening records	36 months	Daily incremental + weekly full	Anonymised after retention; right-to-erasure within 30 days of request
Daily mood logs	24 months	Daily incremental	Auto-archived to cold storage after 12 months
Risk profiles	36 months	Daily incremental	Trend data compacted to monthly aggregates after 12 months
Counselling sessions	60 months	Daily incremental + weekly full	Legal hold possible; session notes encrypted separately
Audit logs	84 months (7 years)	Weekly full	GDPR Article 30 compliance; never deleted; hash chain validated monthly
Consent records	Until revoked + 36 months	Daily incremental	Revocation triggers anonymisation pipeline within 24 hours
Notifications	6 months	Weekly full	Auto-purged; delivery status only retained beyond 6 months
System config history	Indefinite	Weekly full	All versions archived for rollback; never deleted

Backup Strategy: Daily incremental backups to a separate institutional NAS. Weekly full backups tested monthly via restoration drill. RPO $\leq$ 24 hrs; RTO $\leq$ 4 hrs. Backups encrypted with AES-256 and stored off-node.

5 ROLE-BASED ACCESS CONTROL

Resource / Operation	Student	Counsellor	Guardian	Admin	IT Officer
Own screening records (read)		–	–	–	–
Own mood logs (read/write)		–	–	–	–
Own risk status (read)		–	Summary only	–	–
Book counselling session		–	–	–	–
All at-risk student list	–		–		–
Full student profile	–		–		–
Schedule / manage sessions	–		–	–	–
Acknowledge / action alerts	–		–		–
Manage user accounts	–	–	–		–
Configure risk thresholds	–	–	–		–
Export institutional reports	–	–	–		–
View audit logs	–	–	–	–	
Configure RBAC policies	–	–	–	–	
Compliance monitoring	–	–	–	–	
Data retention / deletion	–	–	–	–	

- **Authentication:** Flask-Login with bcrypt (work factor 12) for password verification. PyJWT issues short-lived session tokens (30-min inactivity TTL). 2FA via TOTP (mandatory for Admin and IT Officer; optional for Student).
- **Authorisation:** @role_required Flask decorator checks the role claim in the session token before every protected endpoint. A second DB-level check via RBACPolicy table prevents privilege escalation.
- **Data Isolation:** All SQL queries include a WHERE studentId = current_user.id clause enforced at the repository layer; no raw SQL exposed to controllers.
- **Guardian Access:** All guardian-facing endpoints verify ConsentRecord.granted = True before returning any data. Revocation propagates immediately via a Redis invalidation key.

6 CONTROL FLOW MECHANISM

6.1 Primary Control Patterns

Pattern	Applied Component	Purpose
Event-Driven	Risk classification triggers Observer notifications	Decouples risk detection from alert delivery; new consumers subscribe without code change
Request-Response	All REST API endpoints via Flask + Nginx	Standard HTTP synchronous pattern for dashboard loads, form submissions
Async Task Queue	Celery + Redis for email, reports, reminders	Offloads long-running tasks (email dispatch, PDF generation) from request thread
Scheduled Polling	Celery Beat for weekly SIS sync and batch risk analysis	Ensures continuous data freshness without user-triggered actions
Strategy (runtime swap)	RiskController ↔ IRiskScoringStrategy	Admin can switch ML scorer vs rule-based scorer without restart
State Machine	CounsellingSession status transitions	Enforces valid lifecycle: Requested → Confirmed → {Completed, Cancelled, NoShow}

6.2 Workflow Flowcharts

6.2.1 Workflow 1: Student Screening and Risk Classification

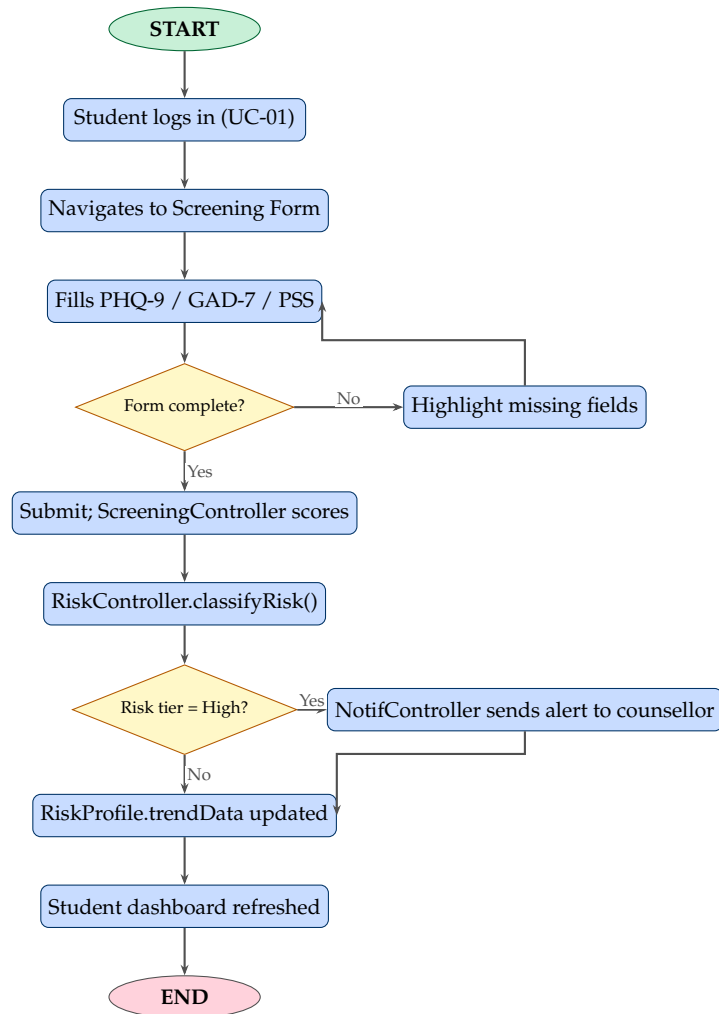


Figure 8: Workflow 1: Student Screening and Risk Classification Flow

6.2.2 Workflow 2: High-Risk Alert and Counsellor Response

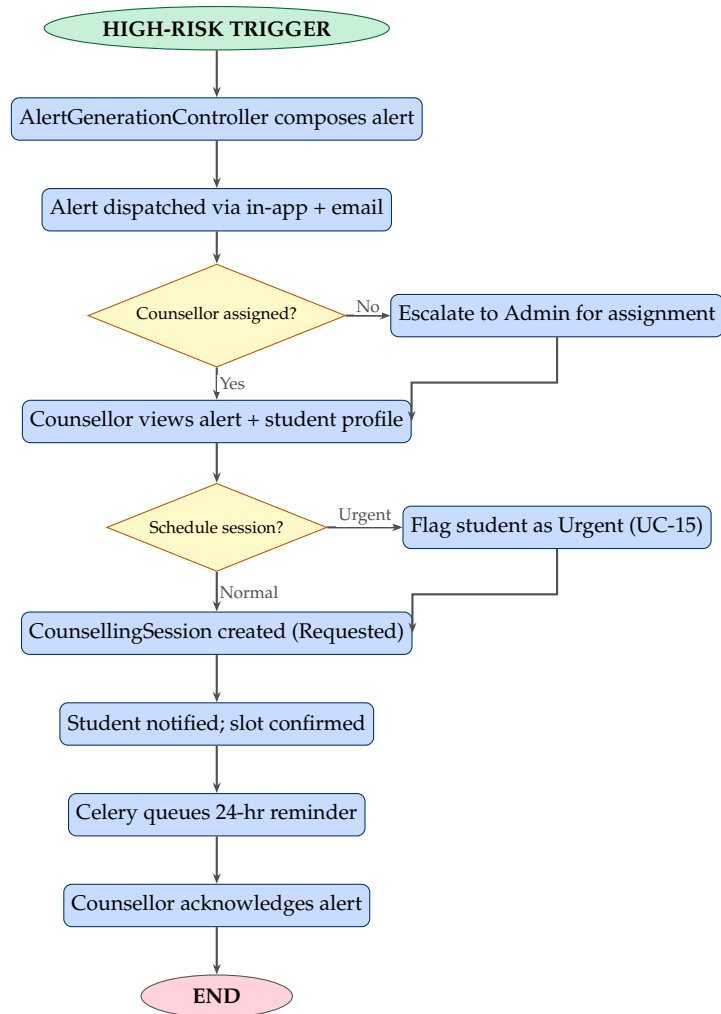


Figure 9: Workflow 2: High-Risk Alert and Counsellor Response Flow

6.2.3 Workflow 3: Admin Institutional Report Generation

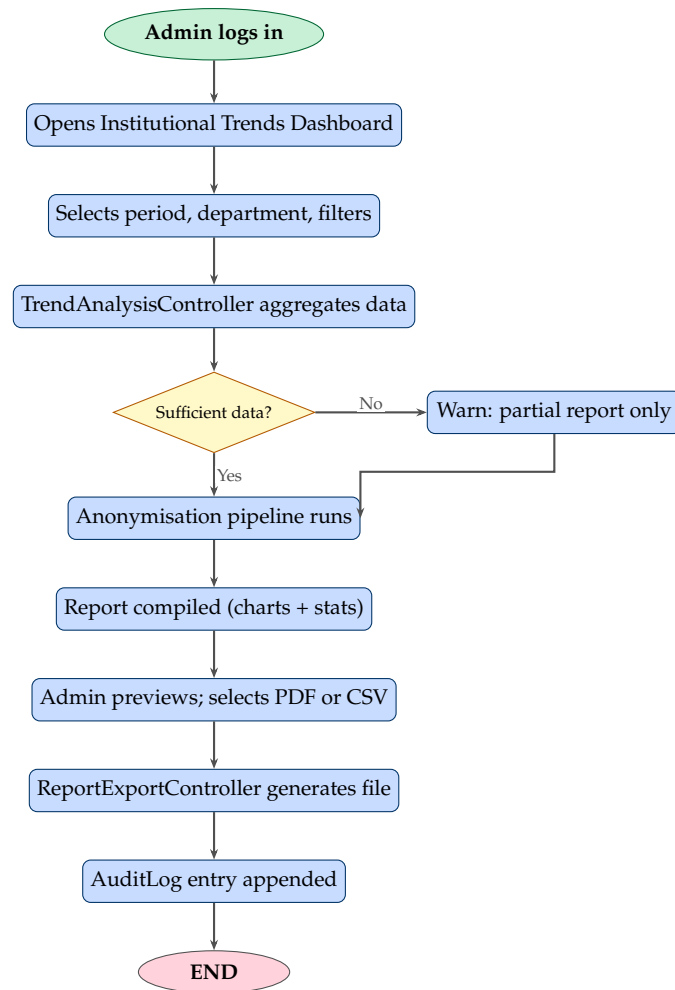


Figure 10: Workflow 3: Admin Institutional Report Generation Flow

6.3 Concurrency and Locking

Scenario	Mechanism	Details
Concurrent screening submissions	PostgreSQL row-level locks	SELECT ... FOR UPDATE on ScreeningRecord prevents duplicate version increments under concurrent form saves
Session slot double-booking	PostgreSQL unique index	UNIQUE (counsellorId, slotDateTime) with status = Confirmed enforces no double-booking at DB level
Alert deduplication	Redis atomic SETNX	Redis key alert:{studentId}:{tier} set atomically; expires after counsellor acknowledgement; prevents duplicate emails
Celery task deduplication	Redis task lock	Each Celery task acquires a named Redis lock before execution; duplicate tasks (same student, same week) are discarded
Dashboard cache invalidation	Redis pub/sub	Risk reclassification publishes an invalidation event; dashboard cache keys are flushed for affected counsellor within 1 sec
Rate limiting	Redis sliding window	INCR + EXPIRE pattern: max 5 failed logins per 15-min window per IP; 100 API requests/min per authenticated user

7 BOUNDARY CONDITIONS

7.1 System Startup

1. **Infrastructure up:** Docker Compose starts containers in dependency order: PostgreSQL 15 → Redis 7 → Flask App → Nginx.
2. **Database initialisation:** Flask startup runs `db.create_all()` (SQLAlchemy); Alembic migration checks schema version; applies pending migrations if any.
3. **Seed validation:** System verifies that at least one Admin account and the default RBACPolicy records exist; creates them from seed if missing.
4. **ML model loading:** MLModelScorer loads the latest `.pkl` model from `/app/models/`; logs `modelVersion` to `PatternReport`. If loading fails, system falls back to `RuleBasedScorer` and logs a `WARN` to the application log.
5. **Celery Beat scheduler:** Celery Beat connects to Redis and schedules recurring tasks: weekly SIS sync (Monday 02:00), daily backup trigger (01:00), monthly report generation (1st of month 03:00).
6. **Health check:** Nginx health endpoint `/health` returns HTTP 200 only after Flask, PostgreSQL, and Redis are all reachable. CI/CD waits for this before marking deployment successful.
7. **Audit log:** A `SYSTEM_START` event is appended to `audit_logs` with timestamp and container version hash.

7.2 System Shutdown

1. **Graceful signal:** `docker compose stop` sends `SIGTERM` to Gunicorn; Gunicorn allows in-flight requests up to 30 seconds before `SIGKILL`.
2. **Celery drain:** Celery workers receive `SIGTERM`; finish the currently executing task; re-queue any un-started tasks back to Redis before exiting.
3. **Session persistence:** Flask-Login clears in-memory session cache; JWT tokens in Redis are left to expire naturally (no active flush required).
4. **Database checkpoint:** PostgreSQL receives `SIGINT` (fast shutdown); runs a checkpoint to flush dirty pages to disk before terminating.
5. **Redis persistence:** Redis performs a final `BGSAVE` before shutdown to persist session and queue state to AOF log.
6. **Audit log:** A `SYSTEM_STOP` event is appended to `audit_logs` with timestamp and initiating actor identity (`SYSTEM` or Admin username).
7. **Container stop:** Nginx container stops last; final access log flushed to disk.

7.3 Exceptional Input Handling

Exception Scenario	Detection Point	Handling Strategy
Invalid login credentials	<code>AuthController.login()</code>	<code>failedAttempts</code> incremented; generic error returned (no credential leak); account locked at 5 attempts; <code>AuditLog</code> entry appended
Incomplete screening form	<code>ScreeningController.save()</code>	HTTP 400 returned; missing field IDs listed; form saved as partial-save (<code>isComplete=False</code>); no risk classification triggered
Out-of-range response values	<code>ScreeningRecord.computeScore()</code>	<code>ValidationError</code> raised; record remains in partial-save state; client receives descriptive error with allowed range
SIS API unavailable	<code>DataCollectionController.importData()</code>	Celery retries 3 times (exp. backoff: 1, 5, 25 min); after 3 failures, Admin receives a system alert; CSV import mode activated
Analytics engine failure	<code>RiskController.classifyRisk()</code>	<code>InsufficientDataException</code> caught; student flagged for manual counsellor review; system continues serving other requests via <code>RuleBasedScorer</code> fallback
DB connection timeout	SQLAlchemy connection pool	Connection pool retries 3 times; circuit breaker opens after 5 consecutive failures; 503 response returned; <code>AuditLog</code> entry on recovery
SMTP delivery failure	<code>NotifController.deliver()</code>	Celery retries 5 times (exp. backoff); after all retries, in-app notification remains active; delivery failure logged to <code>NotifRecord.deliveryStatus</code>
SQL injection attempt	Repository layer (SQLAlchemy ORM)	All queries use parameterised ORM methods; raw SQL prohibited; WAF rule at Nginx layer blocks suspicious patterns; <code>AuditLog</code> entry
Consent revocation mid-session	<code>ConsentRecord.revoke()</code>	Redis invalidation key set immediately; next guardian request returns 403; active guardian sessions invalidated within 1 sec
Large bulk CSV import error	<code>UserMgmtController.bulkImport()</code>	Row-by-row validation; failed rows collected and returned in an error report; valid rows committed atomically; partial import is allowed

8 CONCLUSION

This System Design Document specifies the complete architectural blueprint for the Student Mental Health Monitoring System (SMHMS). The design achieves the following:

- **Security-first architecture:** AES-256 encryption at rest, TLS 1.3 in transit, bcrypt password hashing, RBAC enforced at both application and database layers, tamper-evident append-only audit logging with HMAC hash chains.
- **Proactive risk detection:** The five-subsystem decomposition ensures that academic, behavioural, and screening data flows through the Analytics Engine continuously, enabling ML-based risk classification with ≤ 5 -second single-student latency and automated High-risk alerts within 2 minutes.
- **Scalable containerised deployment:** Docker Compose-based containerisation across four containers (Nginx, Flask, PostgreSQL, Redis) provides fault isolation, independent scaling, and reproducible CI/CD deployments via GitHub Actions.
- **GDPR-compliant data governance:** Structured retention policies, consent-gated guardian access, a right-to-erasure anonymisation pipeline, and a 7-year immutable audit trail ensure full institutional compliance.
- **Resilient boundary handling:** Defined startup and shutdown sequences, circuit breakers, Celery retry policies, and graceful degradation (ML $\rightarrow$ rule-based scorer fallback; SIS API $\rightarrow$ CSV fallback) ensure the system remains operational under partial failures.

The design is directly traceable to the 18 use cases in the Analysis Model (Experiment-04), the UCP-based cost estimate (Experiment-05), and the SPMP schedule (Experiment-06), forming a coherent and complete software engineering artefact set.